



AT ALL NETWORKS

Git & Github



Program



AT ALL NETWORKS

Python syntax

Functions

Modularity

Data structures (list, dict, set, tuple)

Git & Github

Algorithms & Complexity

Problem solving



AT ALL NETWORKS

Git

Features

Git Features



AT ALL NETWORKS

Git is a free and **open source distributed version control system** designed to manage projects of any size.

Features

Open Source

Distributed System

Branching

Merging

History Tracking

Staging Area

Security

Collaboration

Cross-Platform Support

Integration DevOps & CI/CD Tools

Git Features



AT ALL NETWORKS

Open Source

Git is **open source** so anyone can contribute to his development, suggest new feature, and fix bugs.

- Large community support with tuto, guides and forums

- Constant updates and improvements

- No licensing cost

Git Features



AT ALL NETWORKS

Distributed Version Control System

Git is fully **distributed** unlike centralized version control systems.

- Developers can work and commit changes offline.
- Redundancy ensure backups across multiple local repositories.
- Operations are faster since most actions are local.
- Multiple branches enables non-linear development.

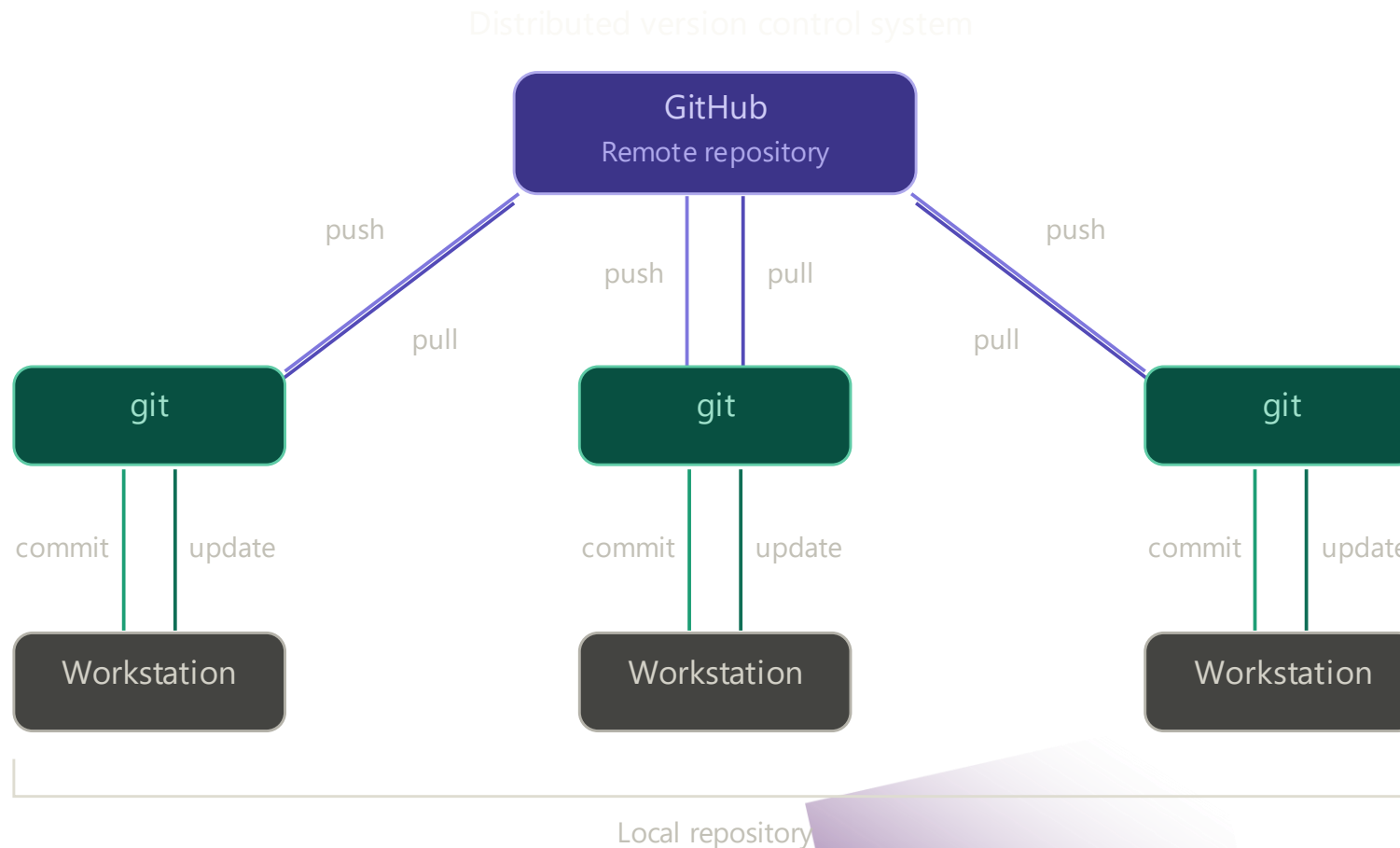
Git Features



AT ALL NETWORKS

Distributed Version Control System

Git is fully **distributed** unlike centralized version control systems.



Git Features



AT ALL NETWORKS

Branching

A **branch** is a parallel **version** of your code, allowing you to work on different tasks without affecting the main codebase.



Usefull to develop new features, experiment, ro fix bugs safely.

Each branch has its own set of commits and changes.

You can merge your branch into the main branch once the work is done.

Branches helps isolating issues, you can fix bugs withouth interfering with development.

Git Features



AT ALL NETWORKS

Branching

A **branch** is a parallel **version** of your code, allowing you to work on different tasks without affecting the main codebase.

Types of branches

Main/Master Branch	Feature Branches	Bugfix Branches	Release Branches	Any other Branches
• The primary stable codebase. • This branch should contain only production ready code.	• For new features.	• For fixing bugs.	• For preparing a version for production.	• Depending of the chosen workflow, you can have many other branches. • Develop / Task / ...

Git Features



AT ALL NETWORKS

Merging

Merging is the process of **combining changes** from one branch into another.



Usually, you merge a feature branch into the main branch after development is complete.

Git tries to automatically integrate changes, but you can have conflicts that needs to be resolved manually.

Git Features



AT ALL NETWORKS

Merging

Merging is the process of **combining changes** from one branch into another

Types of merges

Fast-Forward Merge

- The main branch has not moved forward since the branch was created.
- Git just moves the main branch pointer forward to the latest commit

Three-Way Merge

- Both branches have new commit since they diverged.
- Git creates a new merge commit to combine changes.

Merge conflicts

- Changes on both branches affect the same line of a file.
- Git cannot automatically merge and asks you to resolve conflicts manually.

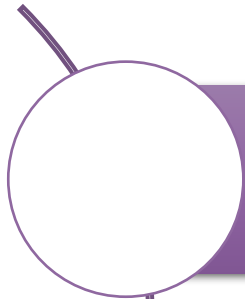
Git Features



AT ALL NETWORKS

Staging Area

It's the place where changes can be reviewed before being committed. You can select commits instead of committing all changes at once.



Usefull for carefull review of code changes.



Helps slitting large changes into smaller commits with more meaning.

Git Features



AT ALL NETWORKS

Security

Git uses **cryptographic hash** functions to ensure the integrity of the codebase. Historically, it used SHA-1, but it is transitioning to SHA-256 to improve security against collision attacks.



Any tempering is detectable because any change in the code changes the hash.

Integrity of commits and repository history.

Git Features



AT ALL NETWORKS

Collaboration

Git allow **multiple developers** to work on the same codebase seamlessly.

- Each developer can clone repositories and work independently.
- Changes can be merged smoothly.
- Platforms like GitHub/GitLab provide pull requests and reviews for effective collaboration.

Git Features



AT ALL NETWORKS

Cross-Platform Support

Git can be used across major operating systems like Windows, Linux, and macOS.



Consistent performance on all platforms.

Used by teams with different systems.

Git Features



AT ALL NETWORKS

Integration with DevOps & CI/CD Tools

Git can be **integrated** in DevOps pipelines and CI/CD tools.

Continuous Integration (CI)

→ Every commit triggers automated builds and tests.

Continuous Deployment (CD)

→ Tested code can be automatically deployed to staging or production.

Branched-based workflows

→ Different branches can trigger different pipelines (dev → Test, main → Production)

Tool Support

→ Works with Jenkins, GitHub Actions, GitLab CI, CircleCI, AWS CodePipeline...



AT ALL NETWORKS

Git

Workflow

Git Workflow – Best practices



AT ALL NETWORKS

Keep main sacred

- Never commit directly to main, it should always reflect production-ready code.
- Every change goes through a branch and a review before landing there.

Work on short-lived feature branches for each task, bug fix, or feature

- Name it clearly and according to the rules of your project/workplace – Ex : feature/user-auth or fix/login-redirect
- Small, focused branches are easier to review and less prone to conflicts.

Commit often, with intention

- Small commits with clear messages make the history readable and debugging easier.
- A good commit message answers "why", not just "what" — fix: prevent crash when user token expires beats fix bug.

Open pull requests early – Don't wait until everything is perfect

- A draft PR lets teammates follow your progress, catch issues early, and share context before the final review.

Synchronize regularly – Rebase or merge from develop branch

- The longer a branch lives without syncing, the harder it gets to integrate.

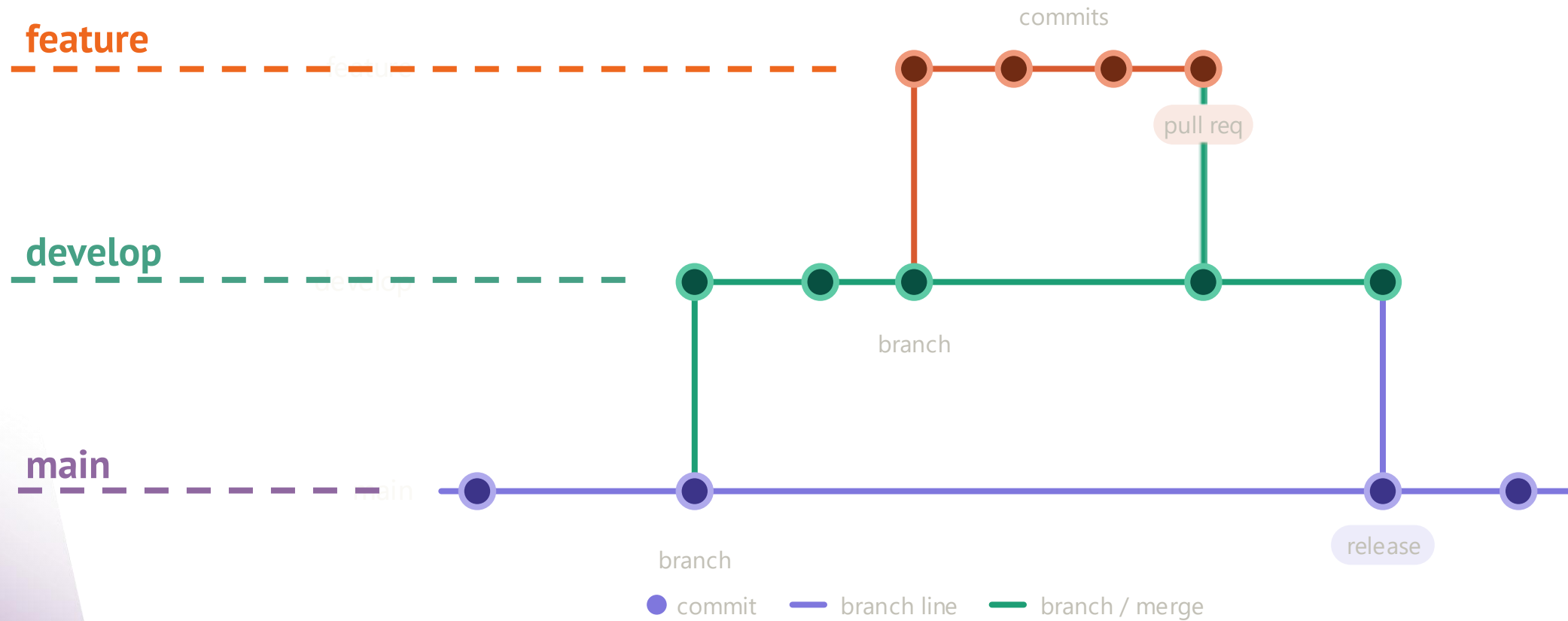
Protect your branches with protection rules

Git Workflow



AT ALL NETWORKS

Branches





AT ALL NETWORKS

Git

Commands

Git Commands



AT ALL NETWORKS

Workflow

Command	Description
git init	Initialize a new local repository
git clone <url>	Clone a remote repository locally
git status	Show working tree status
git add <file>	Stage a file for commit
git add .	Stage all changes
git commit -m "msg"	Commit staged changes with a message
git push origin <branch>	Push local commits to remote
git pull	Fetch and merge from remote
git fetch	Fetch changes without merging
git log --oneline	View compact commit history

Git Commands



AT ALL NETWORKS

Branches

Command	Description
git branch	List all local branches
git branch <name>	Create a new branch
git switch <name>	Switch to an existing branch
git switch -c <name>	Create and switch to an existing branch
git branch -d <name>	Delete a branch (safe)
git branch -D <name>	Force delete a branch
git push origin --delete <name>	Delete a remote branch

Git Commands



AT ALL NETWORKS

Merge

Command	Description
<code>git merge <branch></code>	Merge a branch into the current one
<code>git merge --no-ff <branch></code>	Merge with a merge commit (no fast-forward)
<code>git merge --abort</code>	Abort a merge in progress
<code>git diff</code>	Show unstaged conflicts / differences

Git Commands



AT ALL NETWORKS

Rebase

Command	Description
git rebase <branch>	Rebase current branch onto another
git rebase -i HEAD~n	Interactive rebase on last n commits
git rebase --continue	Continue after resolving a conflict
git rebase --abort	Abort a rebase in progress
git rebase --skip	Skip the current conflicting commit

Stash

Command	Description
git stash	Temporarily save uncommitted changes
git stash pop	Restore the last stash



Exercice de groupe !



AT ALL NETWORKS

We know the way!